Ugreen NAS Admin

User manual

Version 23.8.83 · English

Built from HANDBOOK_EN.md — same visual style as the full layout (colors, frames).

Ugreen NAS Admin – Complete manual for the app and usage

> “Login Track” tab: `## 14. Login Track tab complete`. “NAS management” tab: `## 15. NAS management tab complete` (blue PDF headings like other tabs). Short summaries partly in `HANDBUCH_STRUKTURIERT.md`. Info → Manual opens `HANDBOOK_EN.pdf` — rebuild with `python tools/build_handbook_en_pdf.py`, or newer chapters will be missing.

This edition is tab-centered: Description, buttons, operation, workflows and error cases are directly together for each area - without scattered addendums at the end.

1. Purpose of this manual

From the area: 1. Purpose of this manual

This manual is written as a complete user guide for the current app version. It explains operation tab by tab, including input fields, buttons, typical processes, error patterns and sensible procedures in everyday life. It is not a short description and not a marketing text, but rather a working instruction for real operations. The macOS edition is at [UgreenNASAdmin-mac](https://github.com/runlevel1977-del/UgreenNASAdmin-mac). The Windows edition stays in this project.

Important: The app works directly on your NAS in many areas. Changes are not “simulated” but directly affect files, services, containers and schedules. That's why this manual always describes what a button actually triggers and when you shouldn't use it.

---

2. App logic and security principle

From the area: 2. App logic and security principle

The app has a modular structure: Dashboard, Scripts, Explorer, NAS↔NAS, Devices, Docker, Health, NAS management, Storage, ACL, Snapshots, Backup, Settings. Almost all functions access the NAS via SSH. Without a valid connection, many actions are meaningless.

A central point is the protection mode:

- In Restricted Mode, risky actions are blocked.
- With `Full Rights` critical buttons are released.
- With 'Restrict' you activate the protection again.

The standard for productive environments is:

1. Analyze in normal mode. 2. Only activate full rights for specific interventions. 3. Restrict again after the procedure.

---

3. Header complete (all in one place)

From the area: 3. Header area (header) – current version

Important to classify: The extensive connection fields are in the current UI`Settings`-Tab. The header no longer contains the old large connection block.

3.1 Header elements and effect

- `□ Full Rights` / `□ Restrict`

Toggles risk mode. Critical buttons in multiple tabs depend on this.

- Theme button (`□ Light` / `□ Dark`)

Changes the color scheme of the interface.

- `□ Info`

Opens the info dialog with document buttons (`README`, `Manual`, `CHANGELOG`, YouTube) and below them Check for updates, plus contact area. After a few seconds the app may offer a setup update when a newer GitHub release exists (downloads to the screenshot folder from Settings, otherwise beside the EXE / LocalAppData). Before launching the installer, the app verifies the Ed25519 signature (`.sig`), signed update metadata (installer identity + version), and the SHA-256 digest; on mismatch the file is discarded.

- `□ Screenshot`

Takes a screenshot of the app. Destination folder comes out`Settings -> Pfade -> Screenshot-Pfad`.

- `□Coffee`

Opens the support link.

- SSH status badge

Shows whether the SSH connection is active.

- Model display

Shows the recognized NAS model as soon as it can be read via SSH.

- UGOS/OS line (directly under the model line)

After a successful read, shows among other things `OS_VERSION` and `PRETTY_NAME` from the NAS file `/etc/os-release`; if `OS_IS_BETA=true`, a beta hint is shown. How it fills: (1) after Update everything in the sidebar — `os-release` is read in the same batched SSH round-trip; (2) alternatively on the first start of dashboard live polling (Dashboard tab active, live loop begins) if the line is still empty. Without SSH, a short placeholder text remains.

3.2 What you shouldn't do in the header

- Do not start critical actions if SSH badge is not connected.
- Do not work permanently with full rights.

From range: 22. Full reference: Header buttons (current UI)

22.1 `Full Rights` / `Restrict`

Purpose: Security clearance for critical actions. Prerequisite: conscious confirmation. Effect: Switches many`danger`-marked buttons free or closed again. Typical Error: Permanently released and later accidental deletion/system action.

22.2 Theme button

Purpose: Switch light/dark theme. Effect: purely visual, no NAS change. Typical error: none, just display preference.

22.3 `Info`

Purpose: Open documentation and contact dialog. Effect: No NAS action, only UI dialog.

22.4 `Screenshot`

Purpose: Save current app screen. Requirement: Screenshot target path set sensibly in Settings. Effect: PNG file in the target folder. Typical error: empty/invalid destination path.

22.5 “Coffee”.

Purpose: Open support link. Effect: Browser call.

From the area: 64. Detailed catalog header: every visible button in the context

64.1 Coffee

Lightweight action button in the header. Depending on the app logic, it is usually integrated as a quick trigger or utility function.

64.2 Screenshot

Instantly creates a snapshot of the app interface. Storage location follows that set in Settings`screenshot_dir`.

64.3 Info

Opens the info dialog with:

- Document links (README/Manual/Changelog),
- Project/support links,
- Version/note text.

64.4 Theme / Language

Change appearance and text background. Please note special dashboard rule after language change (German only).`de`, otherwise English).

64.5 Model display

Updates on connected SSH status and shows the detected NAS model.

64.6 UGOS/OS line (below the model)

Adds context: UGOS build (`OS_VERSION`) and base OS (`PRETTY_NAME` / `VERSION_ID` in one line), optional beta flag. Data comes from `/etc/os-release` on the NAS. Refresh: use Update everything (sidebar) or the first start of dashboard live polling if nothing is shown yet.

---

4. Sidebar and navigation complete

From the area: 4. Sidebar and navigation

On the left is the fixed navigation with all main tabs.

4.1 Navigation points

- Dashboard
- Scripts

- Explorer
- NAS ↔ NAS
- Devices
- Docker
- System Health
- Login Track
- NAS management
- Storage
- Users (ACL)
- Snapshots
- Backup
- Settings

4.2 Tool buttons at the bottom of the sidebar

- `Get Pro`

Opens the Pro guide in the main area for Ugreen NAS Admin Pro (separate app). Supporters only (after a PayPal or GitHub Sponsors donation) or selected testers. Flow (5 steps): (1) Donate via PayPal or GitHub Sponsors, (2) receive Pro by email at the address you specified, (3) install Pro and fill in the Activation tab, (4) send activation request with text from the Pro app to `ugna@posteo.de`, (5) receive `pro_entitlement.json` and import the license. The window without a valid license (notice, fingerprint text, and the e-mail/Telegram buttons) is always English, regardless of the installer language. In Pro, Problems & errors reports real issues only (no UGOS/Docker noise, SMART only on FAIL/critical counters, recent script logs only). On the Storage tab: Backup now / Restore handle user data (no `@…` system folders) before/after a disk swap; reinstall apps in UGOS if needed. What Pro can do opens an English browser page with the full capability list (Problems, Logs, Live tail, Console, Dumps, Storage, Overview, Watcher, Technician, Settings; Lab and App development for testers only).

- `Update everything`

Starts an overall refresh of several areas (scripts list, NAS scan, Docker list, health overview, storage tab, …) over SSH. Since v23.8.1 this usually runs as one batched sudo command with markers in the response (fewer round trips). If batching fails, the app falls back to the older sequence of single commands. The same cycle also reads `/etc/os-release` (for the UGOS/OS line in the header) and the list of active `*_serv.service` units on the NAS (for the service combobox in NAS management).

- `Health Snapshot`

Saves the current health status as a report.

From area: 41. Full navigation explanation

The sidebar is not just “tab switching”, but a workflow:

1. `Settings` for basics. 2. `Dashboard` for a quick current overview. 3. `Health` for diagnosis. 4. `Docker` / `Scripts` / `Backup` for operational changes. 5. `Storage` / `ACL` / `Snapshots` for deep file and rights work. 6. `NAS↔NAS` for transfers.

Below:

• `Update All`: synchronizes multiple areas (batched SSH run since v23.8.1, with fallback; also refreshes header UGOS/OS and NAS management → service list).

• `Health Snapshot`: documents state.

Both are very valuable before/after changes.

---

5. Tab Settings complete (fields, buttons, setup, Telegram, SMTP)

From section: 23. Full reference: Settings tab (fields and buttons)

23.1 Main buttons above

• Load: resets form to saved status.

• Apply to current UI: applies form data directly to the running app.

• Save: persists all settings.

23.2 Language area

• Dropdown language: Selection of the UI language.

• Apply language: activates selected language.

23.3 Connection fields

• `NAS IP`, `Port`, `User`, `Password`

• `Use SSH key`

• `SSH key path`

• `passphrase`

• UGOS API: port, HTTPS required, Verify SSL (CA) (dashboard button UGOS API). Default: CA verification off → certificate pin (TOFU). On first contact (or after Forget TLS certificate) a dialog shows the SHA-256 fingerprint — the pin is stored and login allowed only after independent comparison and explicit confirmation (from 23.8.69). If the cert changes: Forget TLS certificate.

• SSH host key: fingerprint dialog on first contact; then pin. Forget SSH host key / Forget TLS certificate.

• Save connection: password, key passphrase and further secrets (e.g. Telegram/SMTP) in the OS vault (`keyring`); settings are written atomically; config/key files on Windows use private ACLs. No plaintext secrets in JSON.

• SSH command: Default (s) and Long (s) — see §79

23.3.1 SSH command timeouts (overview)

Under Settings → Connection, row SSH command:

| Field | Default | Meaning | |-------|---------|---------| | Default (s) | 120 | Short commands (`ls`, `df`, refresh lists) — aborts with a message if exceeded | | Long (s) | 0 | rsync, Top-20 `du`, backup — 0 = unlimited (recommended for large migrations) |

Click Save after changes. Full guide: §79.

23.4 Connection buttons (including new SSH features)

• Save connection: saves all connection fields permanently in config. The SSH password goes into the OS credential vault (not the JSON file). If the vault write fails, secrets stay in the UI and the JSON is not cleared.

• PW Safe: stores only the current SSH password in the system keyring (without the other connection fields).

• Create SSH key pair: creates a new key pair on your PC (`ugreen_nas_admin` + `ugreen_nas_admin.pub`), copies the public key to clipboard, and can directly apply the private key path to the form.

• Install public key on NAS: does a one-time password-based SSH login to the selected target and appends the public key to `~/.ssh/authorized_keys`.

• Forget SSH host key: removes the stored host key (TOFU) for the current IP/port. Needed after a NAS reinstall when connections are rejected due to a changed host key.

• Forget TLS certificate: removes the stored UGOS HTTPS pin for IP/API port. Needed after the NAS certificate changes.

• Profile +/x: create/delete profile.

23.4.0 SSH host key (TOFU)

On first successful SSH contact the app stores the host key in `ssh_known_hosts.json` (with other app data). If the key changes later, the connection is rejected (MITM protection). After a legitimate key change: Forget SSH host key, then reconnect.

23.4.0a UGOS TLS first contact and transport (from 23.8.69)

• Confirm fingerprint: The first TLS fetch only retrieves the certificate (no login). Dialog defaults to No; without confirmation there is no API access.

• HTTPS only: `use_https: false` is rejected before any network call — enable HTTPS and the correct port in Settings.

• SMTP: Credentials require SSL or STARTTLS; on TLS failure there is no plaintext retry. A relay without user+password may still send unencrypted (content visible on the network).

23.4.1 Why this SSH key workflow matters

• Key login removes repeated password entry for day-to-day operations.

• The same private key on your PC can be reused for UGREEN, QNAP, and other Linux systems.

• Automations (script tests, transfers, NAS↔NAS tools) become more stable and faster.

• The password is only needed for initial key installation.

23.4.2 Exact step-by-step flow (UGREEN + second NAS/QNAP)

1. In `Settings -> Connection`, first enter correct host/IP, port, user, and password. 2. Click `Create SSH key pair` and pick a target folder. 3. Optionally set a passphrase:

• with passphrase: RSA 4096 (maximum compatibility),

• without passphrase: preferred Ed25519 (modern/fast).

4. When prompted, apply the private key path into the form and enable `SSH key`. 5. Click `Save connection` so key path and options are persisted. 6. Click `Install public key on NAS` and select target:

• UGREEN: uses the upper connection fields (IP/port/user/password),

• Second NAS/QNAP: uses the second NAS profile below; SSH port is prompted.

7. After successful installation, the app uses the private key for later SSH logins.

23.4.3 What happens technically

- The app installs only the public key on the target system.
- The private key stays on your PC and is never copied to NAS.
- During installation, the app intentionally uses a one-time password SSH session (without key auth).
- On target, one key line is added to `~/.ssh/authorized_keys` (or detected as already present).

23.4.4 What to avoid

- Do not share the private key or copy it to NAS/cloud folders.
- After switching to key auth, do not keep outdated/wrong passwords in the form.
- For QNAP, enter the actual SSH port (not always 22).
- If anything fails, check in order: SSH service enabled, correct user, writable home directory.

23.4.5 UGOS: SSH key persistence (after reboot)

On UGOS, a public key installed via the app may be rejected again after reboot or changes in the UGOS web UI (permissions under `~/.ssh`).

Symptom: Key login worked, then only password again.

Fix (community, MIT): [UGOS_scripts — ssh_public_key](https://github.com/ln-12/UGOS_scripts/tree/main/ssh_public_key) documents a systemd service (`ssh-permission-monitor`) that keeps SSH folder permissions stable. After a successful key install on UGREEN, the app shows a hint with this link.

Note: Scripts there are community content — back up system config before applying; use at your own risk.

23.5 SMB Secondary Profile

Fields:

- Profile name
- Host
- users
- password
- Save password (checkbox)

Buttons:

- Add profile
- Delete profile

23.6 Telegram

- Token + Chat ID fields
- Button for secret privacy (visible/hidden)

23.7 Email

- Host, Port, User, Password, From, To fields
- Checkboxes STARTTLS and SSL

- Secret privacy button

23.7a App development and wishes

In the left bar, directly under Settings.

1. Your name 2. Your email for the reply 3. Kind of wish — new function, change, or both 4. Place in the app — the area on the left, for example Explorer or Docker 5. Your wish — what the app does today and what it should do instead 6. Screenshot — optional, up to four pictures (PNG, JPG, GIF, WEBP, or BMP)

Send wish by email delivers the message to `ugna@posteo.de`. When Settings has an SMTP server, the app attaches the pictures and sends the mail itself. The email program opens only when no SMTP server is saved. The mouse wheel scrolls the page even over the fields. The text box and the picture list scroll themselves only while they still have room.

23.8 Paths

- Scripts path
- Compose path
- Explorer Root
- Screenshot destination path
- `Select folder` for screenshot path

23.9 Script Notify

Fields:

- script
- channel
- Trigger time

Buttons:

- Refresh
- Add
- Delete
- Sync

23.10 Create and install SSH key (complete guide)

This section explains the two new SSH buttons under `Settings -> Connection` so you can complete the full workflow inside the app.

Button: `Create SSH key pair`

- Creates a new SSH key pair on your PC (`ugreen_nas_admin` and `ugreen_nas_admin.pub`).
- Optionally asks for a passphrase:
- with passphrase: RSA 4096 (very compatible),
- without passphrase: Ed25519 (modern, fast).
- Copies the public key to clipboard.
- Can immediately apply the private key path into the connection fields and enable `SSH key`.

Button: `Install public key on NAS`

- Performs a one-time password SSH login (intentionally without key auth for this step).
- Appends the public key to `~/.ssh/authorized_keys` on the target.
- Offers target selection:
- `UGREEN` via top fields (IP/port/user/password),
- `Second NAS / QNAP` via SMB profile below (with separate SSH port prompt).

Why this matters

- After setup, app logins use key auth instead of entering passwords repeatedly.
- The same key pair can be reused across systems (UGREEN, QNAP, other Linux hosts).
- Automations like script tests, transfers, and NAS↔NAS operations become more stable.

Recommended procedure (exact) 1. In `Settings -> Connection`, enter host/IP, port, user, and password correctly. 2. Click `Create SSH key pair` and choose a folder. 3. Optionally set passphrase and confirm key creation. 4. When prompted, apply the private key path into the UI. 5. Click `Save connection`. 6. Click `Install public key on NAS` and choose target system. 7. Confirm success output; later SSH actions run via your private key.

UGOS (persistence): See 23.4.5 — if the key stops working after reboot, set up `ssh-permission-monitor` from [UGOS_scripts](https://github.com/ln-12/UGOS_scripts).

Important rules

- Never share the private key or upload it to NAS.
- Only the public key belongs on target systems.
- For QNAP, always use the actual SSH port (do not assume 22).

From the area: 39. Completely set up Telegram (from zero)

This section fully explains the Telegram setup so that Watcher, Tests and Daily Report work properly.

39.1 Requirements

You need:

- a Telegram account on mobile or desktop,
- Internet access from NAS (for outgoing connections to Telegram API),
- Access the Settings tab in the app.

39.2 Create a bot with BotFather

1. Open Telegram and search for `@BotFather`. 2. Start the chat and send `/start`. 3. Send `/newbot`. 4. Assign a display name for your bot (can be freely chosen). 5. Assign a unique username that ends in `bot` (e.g. `my_nas_alarm_bot`). 6. BotFather responds with a token in the format `123456789:AA...`.

This token is your API key. No shipping without tokens.

39.3 Determine chat ID (easy way)

Variant A (individual chat):

1. Open the chat with your new bot. 2. Send him a message (e.g. `test`) once for the chat to exist. 3. Open in browser: `https://api.telegram.org/bot<TOKEN>/getUpdates` 4. Search for `"chat":{"id":...}` in the JSON response. 5. This number is the chat ID.

Variant B (group):

1. Create or use a group. 2. Add the bot to the group. 3. Send a message to the group. 4. Call `getUpdates` again. 5. Group ID is usually negative (e.g. `-100...`).

39.4 Enter token and chat ID in the app

Settings -> Telegram:

- `Bot Token`: Token from BotFather
- `Chat ID`: from `getUpdates`

Thereafter`Speichern`.

39.5 Test Telegram in Health

Health tab -> Telegram area:

- Press `Telegram test`.
- Check in the target chat whether the test message arrives.

If no message comes:

1. Check token (typo? old token?). 2. Check chat ID (real chat? Group instead of individual chat?). 3. Check NAS internet access (DNS/Firewall). 4. Look for HTTP error text in logs.

39.6 Typical Telegram errors

- `chat not found`

Bot hasn't received message from chat yet or wrong chat ID.

- `Unauthorized`

Token incorrect or revoked.

- Message only comes sometimes

Check cooldown/thresholds in the watcher.

- Group chat doesn't work

Bot not in group or missing group chat ID.

From the range: 40. Complete email/SMTP setup (from zero)

40.1 Minimum values

Settings -> Email:

- SMTP host
- port
- users
- password
- From
- To
- STARTTLS or SSL suitable for the provider

40.2 Typical provider logic

- Port 587: mostly STARTTLS

- Port 465: mostly SSL/SMTPS

Don't activate both blindly. Always set appropriately for the provider.

40.3 Testing

- Run watcher test on NAS (if channel `email` or `both`).
- Send daily report test.

If shipping fails:

1. Check DNS resolution on the NAS. 2. Check SMTP credentials. 3. Check port/TLS/SSL combination. 4. Sender address allowed by provider?

From the area: 65. Detailed catalog Settings: every field explained

65.1 Connection block

- Host/IP: Target address of the NAS
- Port: SSH port
- User: SSH user
- Password/Key: Authentication
- Sudo: required for privileged actions

65.2 Notification block

- Telegram tokens
- Telegram Chat ID
- SMTP Host/Port/User/Pass
- Transmitter/receiver
- TLS/SSL selection

65.3 Path/Tooling Block

- Screenshot directory
- local working directories
- optional tool paths

65.4 Second NAS profile

- Host
- users
- password
- Share/Path information

Benefits: Backup destination and NAS↔NAS.

---

6. Tab Dashboard complete (display, fan, operation)

From Area: 24. Full Reference: Dashboard Tab

24.1 Elements

• Dashboard title

• Live notice (when live polling starts, `/etc/os-release` is read once if the UGOS/OS header line is still empty)

• Webcam button

• Metric tiles (including Network with current interface configuration, filter, and edit controls — see 42.3)

• Fans: toolbar “Probe & map fans…” (discovery/mapping dialog) plus one control tile per discovered fan (1–8, depending on NAS model)

• Docker Live

• scheduled script jobs

24.2 Fans — discovery, mapping, control, and curves

The fan area is not limited to two fixed tiles (system/CPU). The app scans first which fans the NAS reports, then shows as many tiles as there are entries in your mapping (typically 1–4, maximum 8).

Prerequisites (scan and control):

• Tab Settings: NAS IP, user, and password (same SSH as the rest of the Dashboard).

• For write commands (modes, manual PWM, curves, boot profile): enable Full access in the header and accept confirmation dialogs.

• Writable `/proc/it86/fan` (it86 driver) — on some models the interface is missing; the scan log shows this.

24.2.1 Step by step: “Probe & map fans…”

Where: Dashboard → above the fan tiles, button “Probe & map fans…”.

What happens when you open it:

1. The app checks for a NAS IP. Without IP you get a hint — connect in Settings first. 2. A background SSH command with sudo runs (password from Settings). It reads e.g.:

• `/proc/it86/fan` (UGOS-style PWM/status lines),

• `/sys/class/hwmon/hwmon*/fan*_input` (RPM from hardware monitoring).

3. The dialog shows a plain-text log at the top (raw data + meta, e.g. whether `/proc/it86/fan` exists and whether `hwmonitor` is active). 4. Below: table “Discovered fans — mapping” — one row per RPM sensor found (name from scan, e.g. `sysfan1`, `cpufan1`, `fan3`).

What you set per row:

| Column / field | Meaning | What to enter | |----------------|---------|---------------| | Fan name (left) | Tile title | Taken from scan; after Save this is the tile heading | | PWM channel | Which it86 branch is written | Channel 1 (`set` / `cpu`) or Channel 2 (`set2` / `cpu2` / `fan2`) | | RPM line | Which speed is shown live on the tile | Pick the matching sensor name from the scan |

PWM channels: Many UGreen NAS units have two PWM groups but more RPM lines (e.g. three fans, two PWM channels). Several fans may share one PWM channel — one write command then drives multiple physical fans together. Model-dependent; visible in the scan.

Save button:

• Writes the list to `app_settings.json` → `dashboard` → `fan_devices` (array of objects: `id`, `rpm_key`, `label`, `pwm_secondary`).

• Dashboard tiles rebuild immediately — one tile per saved fan.

• Legacy settings (`fan_slot0_*` / `fan_slot1_*`) are replaced on first Save; curves keyed `"0"`/`"1"` migrate to fan IDs on load.

If no fans appear in the table yet: wait for the scan or check the NAS connection. Until the first successful scan, legacy placeholder rows (two lines) may be shown.

24.2.2 Fan tiles — all controls

Each tile matches one `fan_devices` entry. Top of tile: □ name and “Fan (RPM): …” (live value from the chosen RPM line).

| Button / field | Input | What happens on the NAS | |---|---|---| | Silent | click | Fixed PWM ~50% on this fan’s PWM channel; `hwmonitor` stopped first | | Standard | click | UGOS-like auto on that channel + `hwmonitor` restart; intermediate ~128 on it86 | | Max | click | Fixed PWM 100% (255) on the channel | | Manual (%) | dropdown 0–100% in steps of 5 | Display only — effective after Apply | | Apply | click after choosing % | Writes PWM; disables only this fan’s curve; optional boot profile for all manual values (below) | | Fan curve… | opens dialog | Temperature curve for this fan only (section 24.2.3) | | Return UGOS control | first tile only | Returns all fans to UGOS: auto on it86, start `hwmonitor`, remove app boot/curve cron on NAS, disable local curves |

Status line under the buttons: e.g. “OK: …” after successful SSH write or error text (e.g. `/proc/it86/fan` missing).

24.2.3 Fan curve (temperature-controlled)

Open: On the desired tile, click “Fan curve…”.

Dialog — fields and effect:

| Area | Input | Effect | |---|---|---| | Temperature source | CPU (max sensor) or Disk | CPU: highest thermal/hwmon value; disk: SMART temperature of selected drive | | Drive | dropdown `/dev/sda` … | Only for Disk; Load drives fetches list via SSH | | Control points | columns °C and PWM % | At least 2 points, temperature strictly increasing, PWM 0–100 | | + row / − row | click | Extend/shrink curve (window grows with rows) | | Live (NAS) | auto ~every 3s | Current temperature, RPM of mapped fan, calculated curve target % | | Save & activate on NAS | click | Saves curve under `dashboard.fan_curves.<fan_id>`, deploys shared NAS script |

Created on the NAS (when at least one curve is active):

• `/volume1/scripts/ugreen_fan_curve_apply.sh` — applies all active curves

• `/volume1/scripts/ugreen_fan_curve.env` — config with `FAN_COUNT`, per fan `F0_*` … (`ENABLED`, `SENSOR`, `POINTS`, `PWM_SEC`, `STATE`)

• Cron in `/etc/cron.d/papa_jobs` (block “UG-NAS-Admin: fan curve”): every minute + `@reboot` with delay

• Per fan a state file `ugreen_fan_curve.state.<id>` (hysteresis)

Note: Multiple fans can have separate curves; only the fan for which you set manual PWM with Apply is individually removed from curve control. Other active curves keep running.

Boot profile (manual PWM after reboot): After Apply with a fixed percentage, the app may write `ugreen_fan_boot_apply.sh` + `ugreen_fan_boot.env` (`FAN_COUNT`, `F0_PWM`, `F0_USE2`, …) and an `@reboot` entry. Active fan curves and boot profile exclude each other (curve deploy removes boot cron

and vice versa).

24.2.4 Local configuration (`app_settings.json`)

Storage location (portable EXE / one-dir bundle): JSON files such as `app_settings.json` and `nas_admin_connection.json` live in the writable folder next to the EXE (`…/UgreenNASAdmin/`). After switching from one-file to one-dir, the app migrates existing files from the parent `dist/` folder (legacy layout) or from `%LOCALAPPDATA%\UgreenNASAdmin\` on first start when the bundle folder is still empty. Installs under Program Files still use LocalAppData.

Window size and position (from v23.8.42): On startup the main window fits the visible work area (Windows taskbar is taken into account). Size and position are saved under `window` in `app_settings.json` when you exit and restored on the next launch — including on small laptop screens. If the window would be off-screen, it is clamped back automatically.

Under `dashboard`:

```json "fan_devices": [{ "id": "sysfan1", "rpm_key": "sysfan1", "label": "sysfan1", "pwm_secondary": false }], "fan_curves": { "sysfan1": { "enabled": true, "sensor": "cpu", "disk_dev": "", "points": [[40, 25], [55, 45], [70, 75], [80, 100]], "hyst_c": 2 } } ```

- `pwm_secondary: true` = channel 2 (`set2`/`cpu2`/`fan2`).
- Legacy keys `fan_slot0_*` / `fan_curves."0"` migrate on load while no `fan_devices` array exists yet.

Saving Settings: Saving the Settings tab preserves `dashboard` (including `fan_devices`, `fan_curves`, network filter) — not overwritten by empty defaults.

24.2.5 Short workflow

1. Settings: verify NAS connection. 2. Dashboard: “Probe & map fans…” → read scan → PWM + RPM per row → Save. 3. Check tiles (count = discovered fans). 4. Choose mode (Silent / Standard / Max) or Manual % + Apply — with Full access. 5. Optional: Fan curve… per fan. 6. To hand back to UGOS: Return UGOS control (first tile).

Typical issue: one tile only / “unavailable” — repeat scan; check RPM line in mapping dialog; one tile with a single physical fan is normal.

From the area: 42. Dashboard complete: All visible functions in operation

42.1 CPU/RAM cards

Purpose: quick load diagnosis.

Interpretation:

- Short peaks are normal.
- permanently high load + high RAM pressure = check deeper in Docker/Health.

42.2 Volume Cards

Show occupancy and help early in “plate full” scenarios.

When occupancy increases:

1. Open Storage tab. 2. Determine top consumption. 3. Check backup/log/container paths.

42.3 Network
```

Throughput (sparklines) Below the configuration block: per physical NIC estimated RX/TX rates (from `/proc/net/dev`, from the second measurement onward). Useful for backup windows, NAS↔NAS transfers, and Docker pulls.

Current configuration (from the NAS) While the Dashboard tab is active and SSH works, the app runs `ip -j addr` and `ip -j route` on the NAS and shows for the selected interface, among other things:

- operational state, MAC (if reported by `ip`),
- IPv4 and prefix length,
- whether the address appears DHCP/dynamic (JSON `dynamic` field) or typically static,
- whether this interface carries the default IPv4 route and which gateway applies.

This is the live state at sample time — it may differ from what UGOS stores persistently (the NAS UI may use its own DB/files).

Choose interface The Interface dropdown selects which NIC is described in the Current configuration text. The choice is saved locally in `app_settings.json` as `dashboard.net_detail_iface`.

Limit sparklines to selected NICs The field under the hint “Sparklines: empty = all; or e.g. eth0,eth1” only filters the green throughput sparklines — not what exists on the NAS. Empty = all physical interfaces as before. Comma-separated names (semicolon allowed) = only those interfaces in the sparklines, if present. Save filter writes `dashboard.net_monitor_filter` in `app_settings.json`.

Changing values (advanced) Fields: IPv4, prefix (e.g. `24`), gateway, mode:

- Static (ip): applies runtime settings with `ip -4 addr` / `ip -4 route` (via sudo). After reboot, UGOS or a service may override again.
- DHCP renew: on the NAS runs `dhclient` or `dhcpcd` for the chosen interface when available.

Load from NAS fills IPv4, prefix, gateway, and mode from the last received live sample for the currently selected interface (fills the fields only — nothing is changed until Apply).

Apply (sudo) needs Full access in the header (same danger gate as other risky actions) and shows confirmation dialogs. Wrong IP or gateway can drop your SSH session — use only in maintenance windows; keep serial/KVM/console in mind.

Settings and `app_settings.json` When you Save in Settings, `dashboard` and `docker_update` are preserved in the file so Dashboard preferences are not wiped — including `net_detail_iface`, `net_monitor_filter`, `fan_devices`, and `fan_curves`.

42.4 Docker tile

Shows running containers as a leading indicator. If there is a discrepancy, switch to the Docker tab and check the list/logs.

42.5 Script Jobs tile

Shows scheduled jobs and ongoing activities. Helps with correlation “load increase <-> cron job time”.

42.6 Fan area (complete — dynamic discovery)

Layout: Above the tiles, button “Probe & map fans…” and a hint line. Below, a tile grid (two columns): one tile per `fan_devices` entry, not a fixed pair.

Discovery workflow (dialog)

1. Open Dashboard (tab must be active for live RPM; scan works independently). 2. Click “Probe & map fans…”. 3. Wait until the log is filled and “Discovered fans — mapping” shows rows. 4. Per row:

- Choose PWM channel (channel 1 or 2 — see 24.2.1).
- Choose RPM line (name from scan).

5. Save — tiles appear/update immediately.

One fan on the NAS: After scan and Save you get one tile. That is correct — do not expect two.

Four RPM lines, two PWM channels: Four table rows, four tiles; fans sharing a PWM channel are driven together when you write.

Manual control workflow (per tile)

| Step | Action | Result | |------|--------|--------| | 1 | Header → Full access | Write buttons enabled | | 2 | Choose mode or Manual % | Preselection only | | 3 | For manual: pick % in dropdown, then Apply | PWM on NAS; this fan's curve off; optional boot env for all tiles | | 4 | Read status line | "OK: …" or error | | 5 | Observe noise/RPM 1–2 min | — | | 6 | If needed Return UGOS control (first tile) | Full handback to UGOS |

Modes in detail:

- Silent: ~50% PWM, `hwmonitor` stopped briefly — fixed speed.
- Standard: `hwmonitor` active again, it86 auto + reference — closer to UGOS default on this channel.
- Max: 100% PWM.

Fan curve workflow (per tile)

1. Fan curve… on the target tile. 2. Choose temperature source (CPU or disk); for disk use Load drives and pick `/dev/sdX`. 3. Enter control points (e.g. 40°C → 25%, 80°C → 100%). 4. Check Live (NAS) (temperature, RPM, target %). 5. Save & activate on NAS — cron + script on NAS; other fans' curves stay active.

NAS files and cron: see 24.2.3.

Return UGOS control

Visible on the first tile only, but global:

- `hwmonitor` unmask/start
- it86 to `auto` (channels 1 and 2)
- Removes app boot/curve scripts and cron on the NAS
- Sets all local `fan_curves.*.enabled` to `false`

Recommended order on a new NAS model:

1. Save mapping once (24.2.1). 2. Test Standard briefly. 3. Then use curves or fixed PWM. 4. After tests Return UGOS control.

42.7 UGOS API — storage tile

In addition to SSH-based volume cards (42.2), the dashboard shows a "Storage (UGOS API)" tile — same data source as the UGOS web UI (not SSH).

Prerequisite: Settings → Connection → UGOS API (HTTPS port, credentials, SSL). Package `cryptography` for API login.

Content (when the API is reachable):

- Header: model, UGOS version, uptime, optional serial number.
- Pools (left) and physical disks (right) — usage, RAID type, sync status, pool members.

- Extra line: fan RPM (API), network interfaces with link speed (e.g. 1000 Mbit/s), volume usage in percent.

If the API is unreachable: message in the tile; SSH live data (CPU, RAM, network sparklines) continues. The UGOS API button (connection test / snapshot dialog) still provides a full dump.

Full pool/disk report: Storage tab → Pools (UGOS API) (§16, 50.4). Health refresh with UGOS block: §13, 49.3.

From the area: 62. Practical guide: fans including curve and return to UGOS

Part A — Initial setup (once per NAS or after hardware change)

1. Tab Settings: enter IP, user, password; test connection. 2. Open tab Dashboard. 3. Click “Probe & map fans…”. 4. Read the log: is `/proc/it86/fan` present? Are RPM names listed? 5. In the mapping table for each fan:

- System fan often channel 1, CPU fan often channel 2 (if unsure, test both with Silent).
- RPM line = the name that shows speed in the log.

6. Save — tile count = row count. 7. Optional: note current fan speed in UGOS UI for comparison.

Part B — Fixed speed (manual)

1. Header: enable Full access. 2. On the desired tile choose Silent, Standard, Max, or Manual % + Apply. 3. Check status line for “OK”. 4. Observe acoustics/RPM for 1–2 minutes. 5. When done: Return UGOS control or pick another mode.

Part C — Temperature curve

1. Full access on. 2. Fan curve… on the target fan. 3. Choose CPU or Disk; for disk Load drives and pick `/dev/sdX`. 4. At least two control points (rising °C). 5. Watch Live (NAS) — does target % match expectation? 6. Save & activate on NAS. 7. Re-check after a few minutes (cron runs every minute). 8. Avoid manual mode on the same fan — Apply disables its curve.

Part D — Return to UGOS

1. Return UGOS control (first tile). 2. Wait (up to ~1 minute). 3. In UGOS optionally switch fan profile Standard/Silent to verify.

Error “unavailable” / no tiles

- Repeat scan; check SSH/sudo.
- If scan shows 0 sensors: model without it86/hwmon fan — app control not possible.
- If scan OK but RPM empty: wrong RPM line in mapping — reopen dialog and fix.

Error “UGOS does not respond” after return

- Run Return UGOS control again.
- Tab System & Health: check `hwmonitor` status.
- Wait briefly — control does not always change visibly at once.

---

7. Tab Scripts complete (editor, test, operating sequence)

From range: 25. Full reference: Scripts tab

25.1 Left action buttons

- Backup scripts locally
- Update
- TestHost
- Test Docker
- New file
- Delete
- Schedules
- PowerShell/SSH

25.2 Editor buttons

- Template `rsync`
- Template `restic`
- Template `rclone`
- Save (root)
- Save (user)

25.3 Fields

- filename
- Editor content

25.4 Log

- Runtime/error output for test and save.

From the area: 43. Scripts tab complete: Every action in detail

43.1 `Backup` (back up scripts locally)

Purpose: local copy of NAS scripts.

To use:

- Version backup before changes.
- quick recovery.

43.2 `Update`

Reloads script list from NAS.

Always press before editing if changes could have been made at the same time.

43.3 `Testing (Host)`

Executes selected script directly on the NAS.

To use:

- realistic running test without cron.
- fastest way for syntax/path errors.

43.4 `Testing (Docker)`

Tests script in Docker test context.

Useful if the script is actually supposed to run as a Docker job later.

43.5 `New File`

Empties processing state for new script start.

43.6 `Delete`

Removes script. Critical.

Previously:

1. Check file name. 2. If necessary, make a local backup.

43.7 `Schedules`

Opens scheduler drawer and applies target script.

43.8 `PowerShell/SSH`

Manual debug/admin access.

43.9 Templates (`rsync`, `restic`, `rclone`)

Quick start for typical backup scripts.

Always check parameters after inserting:

- Source path
- Target path
- Credentials
- Excludes

43.10 Save (root/user)

`root`:

- for system-level paths if rights are necessary.

`user`:

- for less privileged processes.

Never save unclearly. Understand the target path and rights context beforehand.

---

8. Scheduler complete (all fields, cron, practice)

From range: 26. Full reference: Scheduler Drawer

26.1 Input fields

- minute
- Hour
- day
- Month
- weekday

- First week (checkbox)

26.2 Execution buttons

- As a host job
- As a Docker job

26.3 Display

- Plain text time description
- Target script info

From the area: 44. Scheduler complete: fields, interpretation, best practice

Cron fields:

- minute
- Hour
- day
- Month
- weekday

Option:

- first week (for specific monthly logic)

Buttons:

- Host job
- Docker job

Best practices:

1. First define the time technically (“3:30 a.m. daily”). 2. Set fields. 3. Plain text control in the drawer. 4. Write a job. 5. Note cron post check.

From the area: 63. Practical instructions: Create and check the cron job properly

1. Scripts tab, open target shell script. 2. Open `Schedules`. 3. Set cron fields. 4. Read plain text. 5. Write `Host-Job` or `Docker-Job`. 6. Evaluate cron post check. 7. Check job list. 8. Test manually once.

If job doesn't run:

- Check path permissions.
- Check interpreter (`#!/bin/bash` or `python3`).
- Set environment variables explicitly.

---

9. Tab NAS Explorer complete

From area: 27. Full reference: Explorer tab

27.1 Toolbar

- Scan NAS

- Upload
- Perms 755
- Delete NAS
- Delete PC
- PC->NAS
- NAS->PC
- Search

27.2 NAS context menu

- Load into editor
- Perms 755
- Copy path
- Copy to…
- Move to…
- Upload files
- Upload folder
- Delete

27.3 PC context menu

- Open in Explorer
- Copy path
- Delete

27.4 Additional PC buttons

- drives
- High
- Select folder
- Update locally

From the area: 45. Explorer complete: Work safely

45.1 `Scan NAS`

Reloads NAS file tree.

45.2 `Upload`

Loads local files to NAS.

Actively select the target folder on the left beforehand.

Security: Upload replaces each file only after a verified transfer. Both single-file and ZIP uploads preserve supported ACLs/xattrs. Unknown security attributes abort replacement and keep the original; use a new filename and review permissions. ZIP is atomic per file, not for the entire batch. See [metadata policy (German)](UPLOAD_METADATA_DE.md).

45.3 `Perms 755`

Sets rights to target path.

Only use if it is clear why.

45.4 `Delete NAS`

Deletes selected NAS objects.

Critical: Double check focus and selection.

45.5 `Delete PC`

Deletes selected local files.

45.6 `PC -> NAS` and `NAS -> PC`

Transfer between right and left side.

Always check both paths visually before transferring.

45.7 Search

Searches in the current NAS context.

45.8 Context menus

The context menus are equivalent action triggers to toolbar buttons, not just ads.

NAS → NAS (right-click): Select files/folders in the left tree → right-click → Copy to… or Move to… → pick the destination from the list (full path, e.g. `/volume2/Papa Sicherung/APPS`, not `/volume2/APPS`). If a short path uniquely matches a folder in the tree, the app offers it. System folders (`@…`, recycle bin) and volume roots as source are blocked. If the name already exists at the destination, the action stops. Requires full access.

---

10. Tab NAS↔NAS complete

From range: 28. Full reference: NAS↔NAS tab

28.1 Top buttons

- Scan Ugreen
- (Windows) SMB scan

28.2 Profile/Status Area

- SMB profile combo
- SMB status label

28.3 Context menus

- Upload to the other side
- Delete on current page

From the range: 46. NAS↔NAS complete: SMB workflow

46.1 Requirements

- Second NAS profile in Settings complete.

- (Windows) SMB mechanism available.

46.2 Process

1. Scan Ugreen. 2. Scan SMB/Shares. 3. Select target paths left/right. 4. Trigger transfer via context menu. 5. Check result/error.

46.3 typical errors

- Host not reachable.
- Auth wrong.
- Share hidden/not shared.
- Rights for target path are missing.

---

11. Network devices tab complete

From area: 29. Full reference: Device tab

29.1 Action

- Search devices

29.2 Result

- Table with child/name/IP/detail
- Status label (`scanning`, `empty`, error text)

From the area: 47. Device tab complete

`Geräte suchen`starts Discovery via NAS-SSH. Display contains LAN/USB information.

Error case:

- without SSH no data.
- If there is a discovery error, status output appears.

---

12. Tab Docker complete (buttons + operation directly together)

From range: 30. Full reference: Docker tab

Runlevel apps (Transfer Hub / Wake & Sync): The folder picker shows local NAS paths including Home (`/home/...`, e.g. Photos) without an SMB share. Only system folders (`@...`) stay hidden. Transfer Hub 0.6.28+: multi-select source folders (check boxes → "Add checked") — one transfer per folder; destination stays a single folder. After updating the UPK apps, restart/reinstall (Transfer Hub 0.6.28+, Wake & Sync 0.1.32+).

UGOS install/config (Transfer Hub 0.6.39+ / Wake & Sync 0.1.50+): Choose up to three folders — required UGOS folder plus optional Extra UGOS folder 2 and 3 (other volumes allowed). Leave unused extras empty; restart the app after changes. First app password always asks for New + Confirm (Security Hub 0.1.22+ as well).

Transfer Hub 0.6.41: The log uses the language of the interface. That choice also applies to scheduled runs. Lines that rsync or rclone print themselves stay in English.

UGOS security update, October 2026: Transfer Hub 0.6.40 checks local source and destination paths when a profile is saved and again immediately before every run. Each expanded route, including the path passed to `rsync`, must stay inside the selected volume after symlink resolution. `..` and symlinks that leave the volume are rejected. Packages from 0.6.36 only had part of this check. Security Hub 0.1.21 shows sign-in names in Login Track as text, not HTML. Wake & Sync 0.1.46+ sets the first app password only after the UGOS administrator who installed the app signs in. Bug reports: Technical Support on the app page, or `runlevel1977@posteo.de`.

30.1 Creation/Catalog Group

- Build Docker
- Docker catalog
- New Docker
- Docker update

30.2 Management Group

- Exclusion list
- list
- Stop all containers
- start
- Stop
- Restart
- List (second position in the layout)

30.3 Diagnostic group

- Stats
- Inspect
- Delete
- Fix 777

30.4 Log/Compose area

- Live log start
- Live Log Stop
- Compose config
- Compose ps
- Compose up -d

30.5 fields

- Compose file path
- Container treeview selection

From the area: 48. Docker complete: operation, updates, diagnostics

48.1 Create/Catalog

- Build Docker
- Docker catalog
- New Docker

Use Catalog for image search; pick Compose for stacks with UGREEN paths. Ready-made presets include:

| Search / image | Purpose | Typical ports | |---|---|---| | MeTube (`alexta69/metube`, `ghcr.io/alexta69/metube`) | Video/audio download (yt-dlp, web UI) | 8081 | | Scrutiny | SMART disk monitoring | 8080 | | Dozzle | Docker container logs in browser | 8080 | | Jellyfin / Plex | Media streaming | 8096 / 32400 | | Uptime Kuma | Uptime monitoring | 3001 | | Homepage | Homelab dashboard | 3000 | | Portainer | Docker GUI | 9000 | | Sonarr / Radarr / Prowlarr | Media automation | 8989 / 7878 / 9696 |

Presets use volumes under `/volume1/docker/<name>/…` — adjust paths in the wizard, then deploy.

Missing from App Center (15 recipes): Docker tab → Missing from App Center — curated stacks for services not in the UGOS App Center (MeTube, Jellyfin, Immich, Paperless-ngx, Vaultwarden, Nextcloud, AdGuard Home, Sonarr/Radarr/Prowlarr, qBittorrent, Uptime Kuma, Home Assistant, Syncthing, Portainer). Search by name/tag; double-click or Open in wizard. Detailed step-by-step guide: §77.

Homelab stacks (multi-service compose): Docker tab → Homelab stacks → e.g. monitoring (Uptime Kuma + Dozzle + Homepage) or media download (MeTube + qBittorrent).

Compose from GitHub: Docker wizard → From GitHub… — URL to `raw.githubusercontent.com` or `github.com/…/blob/…/docker-compose.yml`.

UGOS API (dashboard): UGOS API button — live data via the web UI API (same as the UGOS app), not SSH. Port/HTTPS: Settings → Connection (default HTTPS port 9443). Requires `cryptography` for login (`pip install cryptography`).

SSH timeouts / exit codes: Settings → Connection → SSH command — §79. Migration incl. pre-flight: §78. Storage Top-20: §80.

48.2 Update and mass actions

- Docker update (selective)
- list
- Exclusion list
- Stop everything

Selective updates are more production-safe than global.

48.3 Runtime Control

- start
- Stop
- Restart

Always work with a fresh list.

48.4 Diagnosis

- Stats (resources)
- Inspect (structure)

- Logs (historical/live)

48.5 Compose

- Enter compose file
- check config
- ps check
- run up -d

Before`up -d`always double check config.

48.6 Critical action `Delete`

Clarify beforehand:

- Where is persistent data located?
- Recovery path available?
- dependent services affected?

From the range: 60. Practical guide: Safely update Docker service

Goal: An existing container should be updated without uncontrolled side effects.

1. Open Docker tab. 2. Click on 'List'. 3. Mark target container. 4. Call `Inspect` and note critical information (volumes, ports, Env). 5. Open `Logs` and note the baseline. 6. Run `Docker Update`. 7. Check the status again with `List` and `Inspect`. 8. Carry out functional testing from an application perspective. 9. In case of errors: stop/start container; if necessary, go back to the previous image.

Why this order is important:

Without a baseline from Inspect/Logs, it is often unclear later whether the error came from the update or was already there before.

From the range: 77. Practical guide: Docker recipes (Missing from App Center)

Goal: Install a service not offered in the UGOS App Center — using a ready-made compose template with UGREEN paths under `/volume1/docker/…`.

Prerequisites

1. SSH connection to the NAS is working (green connection status in the app). 2. Docker is running on the NAS (Docker tab → List shows containers or an empty, error-free list). 3. You know which service you want (e.g. Jellyfin, Immich, Vaultwarden).

Step 1 — Open the recipe library

1. Sidebar → Docker Manager (Docker tab). 2. In the top toolbar click Missing from App Center. 3. A window opens with all 15 recipes and a search field at the top.

Step 2 — Select a recipe

1. Optionally type in the search field (e.g. `jellyfin`, `photo`, `password`, `8081`) — the list filters immediately. 2. Click a recipe — a short description appears below (purpose, port hint). 3. Confirm with Open in wizard (or double-click the list entry). 4. The recipe window closes; the Docker wizard opens with the ready compose YAML in the editor.

Step 3 — Review and adjust compose

1. Read through the YAML — especially:

- ports: host port → container port (e.g. `"8096:8096"`). Avoid port conflicts with other services.
- volumes: paths start with `/volume1/docker/<service>/…` — change to `/volume2/…` if Docker should live on another volume.
- environment: passwords, secret keys, `PUID`/`PGID`, timezone (`TZ=Europe/Berlin`).

2. For Immich and Paperless-ngx: replace `change_me_db`, `change_me_secret`, etc. with secure values. 3. For AdGuard Home: use port 53 only if no other DNS service on the NAS or router blocks that port. 4. Adjust media paths (e.g. Jellyfin `/volume1/media` — folder must exist or is created on deploy).

Step 4 — Scan variables (recommended)

1. In the Docker wizard click Scan variables. 2. Review detected placeholders, volume host paths, and ports in the form. 3. Enable Create host folders on NAS (chmod 755) if the app should create empty Docker folders. 4. Click Next — values are applied to the YAML.

Step 5 — Deploy

1. Review compose (optional config step in the wizard if available). 2. Start deployment (up -d / Start in the wizard — depending on UI version). 3. Wait until the log completes without errors. 4. Docker tab → List — container should be running.

Step 6 — Functional test

1. In the browser open `http://<NAS-IP>:<port>` (port from recipe, e.g. Jellyfin 8096, MeTube 8081, Vaultwarden 8222). 2. Complete first-time setup in the service web UI (admin account, libraries, etc.). 3. If problems occur: Docker tab → select container → Logs and Inspect.

Step 7 — After first start (best practice)

1. Store passwords/secrets securely (password manager — do not leave defaults in compose only). 2. Optional snapshot in the Snapshots tab for the Docker data folder. 3. Document the compose file on the NAS (e.g. save under `/volume1/docker/<service>/docker-compose.yml`).

Recipe overview (ports)

| Recipe | Typical port | Notes | |--------|--------------|-------| | MeTube | 8081 | Downloads under `/volume1/docker/metube/downloads` | | Jellyfin | 8096 | Media folder `/volume1/media` | | Immich | 2283 | Multiple containers + Postgres — set DB password | | Paperless-ngx | 8000 | Redis + Postgres — set secret key | | Vaultwarden | 8222 | No signups (`SIGNUPS_ALLOWED=false`) | | Nextcloud | 443 | linuxserver image, HTTPS | | AdGuard Home | 3000 (+53 DNS) | Check DNS port conflicts | | Sonarr / Radarr / Prowlarr | 8989 / 7878 / 9696 | *arr stack, shared `/volume1` data | | qBittorrent | 8080 | Downloads `/volume1/downloads` | | Uptime Kuma | 3001 | Monitoring dashboard | | Home Assistant | 8123 | Persistent config | | Syncthing | 8384 | Sync folder `/volume1/sync` | | Portainer | 9000 / 9443 | Docker socket access — trusted network only |

Common issues

- Port in use: Pick another host port in YAML (e.g. `"8097:8096"`).
- Permission denied: Adjust PUID/PGID to NAS user or create host folders via the wizard option.
- Container won't start: Read logs; for DB images (Immich/Paperless) wait until Postgres/Redis are healthy.

---

13. Tab System & Health complete

From area: 31. Full reference: Health-Tab

31.1 Main buttons

- Refresh Health
- Check RAID
- Check SMART
- Storage
- Scheduler inventory
- Save report
- Restart
- Shutdown

31.2 UGOS panel

- Title + refresh
- Service status label

Note:

- `Refresh UGOS services` reloads the current state of core NAS services.
- This panel speeds up troubleshooting (for example, quickly seeing if a core service is down before going deeper into Docker/scripts).

UGOS API (live) on health refresh: When the UGOS web API is configured, Refresh Health adds a "UGOS API (live)" block with model/version/uptime, fan RPM, NIC link speeds, and volume usage — complementing SSH-based checks.

domain_tool: If `domain_tool.service` is failed, a warning appears in the health log (SMB/domain configuration). The report also includes error signatures for `domain_tool` / `smbftpd` from journal and log tails.

31.3 Telegram block

Fields:

- Enabled
- interval
- Disk warn/crit
- Temp
- Cooldown
- Fan min

Buttons:

- Telegram test
- Manual check

31.4 Watcher Block

Channel:

- Telegram / Email / Both

Checkboxes:

- Disc
- RAID
- Network ready
- Temp
- docker
- SMART service
- SMB/NFS services
- Maintenance timer
- systemd failed
- fan
- Login failures

Fields:

- Fan min
- Login window
- Login min
- Require containers
- Ignore patterns
- Auto restart names

Buttons:

- Save locally
- Install on NAS
- Test on NAS

Deploy path (from 23.8.56): Watcher, config and state live under `/var/lib/ugreen-nas-admin/` (no longer under `/volume1/scripts/`). After updating, run Install on NAS once; cron lines must use the same path.

31.5 Daily Report Block

- Enabled checkbox
- Save locally
- Install on NAS
- test
- Deploy path (from 23.8.56): Script and config under `/var/lib/ugreen-nas-admin/`. Cron example: `.../ugreen_script_notify_runner.py ... -- /usr/bin/python3 /var/lib/ugreen-nas-admin/ugreen_daily_report.py`. Dry-run: `python3 .../ugreen_daily_report.py --dry-run`; real send test: `--force-send`.
- Report content (NAS script): Since v23.8.1 the daily report includes an OS / UGOS section (excerpt from `/etc/os-release`: e.g. `PRETTY_NAME`, `VERSION_ID`, `OS_VERSION`, `OS_IS_BETA`) — useful for support and version checks alongside the existing blocks.

---

14. Login Track tab complete

From the area: Login Track — access by client IP

The Login Track tab (sidebar icon □, between System & Health and NAS management) shows sign-ins, failed logins, and active connections to the NAS — focused on the client IP (not internal NAS services). The app reads several log and status sources on the NAS over SSH, normalizes lines into entries, and lists them in a read-only log (text widget). Live mode is on by default: you mostly see new events since opening the tab or since turning live on; with Refresh or live off you can load history instead (about 30 days `journalctl` window plus log tails).

14.1 Purpose and typical use

- Who signed in when via SSH, SMB, UGOS app (iPhone/PC), UGOS web, or an open TCP connection?
- Live watch while you test sign-in from phone/PC.
- Review older access after Refresh with live off.
- Sort by date/time, IP, user, source, or outcome; export to a text file; optionally block an IP via the UGOS block list.

Scope: Login Track does not replace the Telegram/email guard on System & Health (thresholds, RAID, temperature). It is a dedicated access log in the app.

14.2 Prerequisites

- NAS IP and SSH in the header / Settings — without a working session you get hints instead of entries.
- Reading logs is enough for display; Block IP writes `/ugreen/.config/block_ip_list` on the NAS over SSH with sudo. The NAS IP and loopback cannot be blocked.
- Leaving the tab stops live polling.

14.3 Tab layout

Top

- Title and subtitle (sources and live mode).
- Refresh — with live on: resets baseline and clears the list (waits for new lines). With live off: one history collect.
- Export… — saves the visible report (headers, sort, diagnostics) as a text file on the PC.
- Block IP… — IPv4 dialog; writes to the UGOS block list (see 14.9).

Sort and filters

- Sort by: Date / time, IP address, User, Source, Outcome.
- Newest / Z–A first — for date/time, checked = newest on top; unchecked = oldest on top. Date/time sorts by calendar day, then time of day; rows without a parseable timestamp stay at the bottom (even when descending).
- Live (since tab start only) — default on. Baseline on first poll; noted in the header diagnostics.
- Hide app session pings — default on. Hides repeated UGOS session/VerifyToken noise; keeps real logins.

List

- Read-only (keyboard edit blocked); select text and right-click supported.

- Columns: `Time | IP | Source | Outcome | User | Detail`
- Separator lines between entries.
- Header: host, mode, entry count, sort order, diagnostics, SSH notes on errors.

14.4 Data sources on the NAS (technical)

One bundled SSH command returns sections marked `@@SOURCE:...@@` (history vs live sets differ slightly):

| Section | Content (short) | |---------|-----------------| | `ssh_journal` / `auth_log` | OpenSSH accepted/failed/invalid, sessions, disconnect | | `log_serv` | UGOS log_serv.slog login and Samba audit | | `ctl_serv` / `entry_serv` | UGOS app/web login, VerifyToken, biometrics, user-agent | | `gateway_serv` | gateway_serv_gin.slog (login/session related) | | `journal_ctl` | Short journalctl window (live) | | `nas_conn` | `ss`: established TCP to common service ports | | `last` / `lastlog` | Classic last output (history mode) |

The app's own collect command echo is filtered so live view is not flooded with sudo/journalctl noise from this tool.

14.5 Source column (display)

Typical Source values:

- SSH — SSH sign-in / failure / disconnect
- UGOS Samba — SMB per UGOS log
- UGOS iPhone / UGOS PC App / UGOS Web — client from user-agent / module
- UGOS login — other UGOS login lines
- NAS connection — active connection from `ss`
- last — last output

Outcome: e.g. `ok`, `failed`, `info`, `session`.

14.6 Live vs history

| Setting | Behavior | |---------|----------| | Live on (default) | Polling about every 4 s on tab focus. First round = baseline; then deltas only. Toggling live or Refresh resets baseline. | | Live off | Refresh loads history; list kept until next refresh. |

Live filter: Treats `ok`/`failed` or timestamps near tab start (about 2 minutes slack) as new — avoids old journal lines filling the live list.

14.7 Sorting in detail

- Date / time: chronological across mixed formats (`YYYY-MM-DD HH:MM:SS`, ISO with `T`/timezone, journal `Mon DD HH:MM:SS` with inferred year). Stable tie-break on original time string.
- IP: numeric IPv4, then time.
- User / source / outcome: alphabetical, then time.
- Newest / Z–A first: reverses primary order; missing timestamps stay at the bottom.

14.8 Export

- Export… — file dialog; empty list → load first.
- Exports the visible report after filters/sort, not raw SSH.

14.9 Block IP

• Block IP… or right-click a row with an IP.

• Confirmation; writes JSON list `/ugreen/.config/block_ip_list` on the NAS. Duplicate IPs are not added twice.

• Not blockable: 127.0.0.1 and the header NAS IP.

• Effect depends on UGOS/firewall using that list.

14.10 Troubleshooting and tips

• “SSH response missing expected log sections” — connection, permissions, or UGOS paths; read the note block.

• Empty live list — no new sign-in after baseline; test from another client; try Hide app session pings off.

• Missing UGOS app login — some sessions only log verify/is_login; check ctl_serv / log_serv.

• Many SSH lines from this PC — your admin session also logs; use the IP column for remote clients.

• Entry cap in session — large internal buffer; export during long live sessions if needed.

---

15. NAS management tab complete (actions over SSH/sudo)

A dedicated tab between “System & Health”, Login Track, and “Storage & Shares”. This tab is for actions on the NAS, not passive diagnostics. Commands run over SSH with sudo.

15.1 Layout, scrolling, and window size

• Two columns: All controls on the left, a log on the right showing SSH output for each action.

• Scrolling (left): The left column is taller than the viewport — scroll vertically with the mouse wheel while the pointer is over the left column (fields and buttons). The scrollbar on the right edge of the left column still works (drag or click track). Scrolling should feel smooth (scroll region updates with each wheel step).

• Log width: A sash (splitter) sits between the two panes. Drag it to balance width. On tab open, width is set 50:50 between the left pane and log; each button has its own cell so long labels do not stretch an entire row. Widen or narrow the log as needed.

15.2 Prerequisites and “Full access”

• Writes or risky actions (change files, reload services, maintenance, USB eject, SSH profiles, Samba, earlyOOM, NGINX recovery, …): Full access in the header and an SSH user with sudo (same idea as reboot/shutdown in Health).

• Read-only / lists / status only: e.g. USB list, disk list, LED slots, share list, read cron, read `power.conf`, RAID check status — often works without unlocking danger actions, as long as SSH is connected.

• Always read the log for errors; when unsure, do read-only steps first.

15.3 Recommended workflow

1. Read the short intro at the top of the tab. 2. Refresh lists (USB, disks, shares, …), then pick an item in the dropdown. 3. Read confirmation dialogs before any write. 4. After each action, check the log (`systemctl`, `smartctl`, `testparm`, etc.).

15.4 Areas at a glance

| Area | Purpose (short) | Typical NAS targets | |------|-----------------|---------------------| | Power & WoL | After power loss / Wake-on-LAN, HDD spin-down | `/etc/power.conf` (via `crudini`), `internal_disk_sleep` | | UGOS power scheduler | Weekly power off/on like UGOS GUI | `/etc/power.conf` `[poweroff]`/`[poweron]`, `OffSched`, `OnSched` | | Scheduled shutdown | Daily shutdown at fixed time (cron) | `/etc/cron.d/nas_admin_timed_shutdown` | | USB | Safe eject (UGOS) | `USBDiskStop`, `umount`, mounts under e.g. `/mnt/@usb/...` | | SMART | Self-tests and log | `/dev/sd...`, `smartctl` | | RAID & filesystem maintenance | RAID scrub, TRIM, ext4 scrub | `mdcheck`, `fstrim`, `e2scrub_all` (systemd) | | SSH (drop-in) | Extra `sshd` rules with rollback | `/etc/ssh/sshd_config.d/60-ugreen-nas-admin.conf` | | UGOS core services | Start/stop/status, journal, `.slog` tail; list extended on Update everything | `*_serv.service`, `/var/ugreen/log/*.slog` | | Network (UGOS) | Dual-NIC overview, read-only | `/etc/network/ugos.d/*.json`, `ip link`/`ip addr` | | NGINX | Reload or restore ROM config | `ugnginx-reload`, `/rom/etc/nginx`, … | | earlyOOM | Tune low-memory killer | `/etc/default/earlyoom` | | Samba | Shares, empty recycle, quick add | `smb.conf`, `testparm` | | LED & beeper | Chassis identification | `/sys/class/leds/diskN`, `ugbeep` |

15.5 Power & Wake-on-LAN

- Read: Reads `power_boot` and `wake_on` from `/etc/power.conf` and fills the combos; may show raw lines in the log.
- Save: Writes selected values (usually `true`/`false`) with sudo. Only change what you understand (UGOS / NAS docs).
- Write WoL to power.conf: Writes the current Wake-on-LAN selection only (shortcut if you only adjust WoL).
- HDD spin-down: Combo `internal_disk_sleep` (minutes until internal disks spin down, `0` = off). Loaded/saved with the power group from `/etc/power.conf`. Longer idle times save power; very short values can be annoying under frequent access.

15.6 UGOS power scheduler (weekly plan)

Same idea as the UGOS UI: scheduled power off and wake (rtcwake) per weekday via `/etc/power.conf` and UGOS helpers `OffSched` / `OnSched`.

- Fields: Per day (Mon–Sun) power off and power on as HH:MM (24 h). Multiple times per day separated by comma (e.g. `22:30` or `22:00,23:30`). Empty = no entry for that day.
- Checkbox “Scheduled power on/off enabled”: Maps to `enable_scheduled_power` in `power.conf`.
- Load: Reads existing NAS config and fills fields (including HDD spin-down).
- Preview: Shows planned values in the log without writing — recommended before first Apply.
- Apply: Writes `power.conf` via `crudini` and runs `/usr/sbin/OffSched` and `OnSched`. Requires Full access and sudo. Real shutdown — plan maintenance windows.
- Regenerate: Rebuilds scheduler lines in `power.conf` when GUI and file drift apart.

Note: Block “Scheduled daily shutdown” (cron, 15.7) uses a different mechanism (`/etc/cron.d/...`). Both can coexist — check which is active on your NAS before changing schedules.

15.7 Scheduled daily shutdown

- App-managed file: Write cron creates/updates only `/etc/cron.d/nas_admin_timed_shutdown`. If you never used that button, the file may be missing even though UGOS shuts down daily — the schedule might live in another `/etc/cron.d/*` file, root’s crontab, under `/var/spool/cron/...`, or `/etc/crontab` (often the stock NAS UI does not use the app file).
- Read cron prints several blocks: the app file (if present), listing `/etc/cron.d/`, grep for shutdown/poweroff/halt/TimedShutdown (UGOS often uses `/sbin/TimedShutdown` in the root

crontab, not under `/etc/cron.d`), direct reads of spool files (when `crontab -l` is empty over SSH), `/etc/crontab`, and systemd timers. If everything is empty or only generic timers appear, the schedule may exist only in the UGOS GUI or another mechanism — check there. SSH must be connected (otherwise you only see “Not connected”). The first cron line with a shutdown-related keyword (including TimedShutdown) updates the time fields (checkbox on) when minute/hour can be parsed (if multiple weekday lines exist, this reflects the first matching line in the output).

• Enable via this app: HH:MM (24 h), then Write cron as above. Real shutdown — plan maintenance windows.

• Disable here: Removes only the app file (with confirmation) — a schedule configured elsewhere on the NAS may continue until changed there.

15.8 USB (UGOS)

1. Refresh USB list — finds typical USB mount paths. 2. Pick a mount, UGOS eject: shows lsof/fuser; warns if activity is seen; then `USBDiskStop` (if present), `sync`, `umount`. Close apps using the disk first.

15.9 SMART

• Disks: Refresh list, select a block device.

• Test: *short* (minutes), *long* (very long, heavy), *conveyance* — depends on disk/firmware.

• Self-test log: Prints recent SMART / test history from logs or `smartctl` as available on the system.

15.10 RAID & filesystem maintenance

• Start RAID check: Starts the systemd unit used for the scheduled RAID check flow (`mdcheck_start`), as UGOS defines it.

• Status / Progress: Read-only — shows current progress / md-related info.

• fstrim / e2scrub_all: Starts those systemd units — can cause noticeable IO; use during quiet periods.

15.11 SSH hardening (drop-in)

• Profile (*high* / *middle* / *low*): Writes a drop-in under `sshd_config.d`. From 23.8.69 a systemd timer (~4 min) is started and verified before the drop-in is replaced atomically; effective algorithms are checked with `sshd -T`. Syntax/reload failures restore the previous state — no restart fallback.

• Auto-rollback: Uses the systemd timer (`at` not required). Do not reboot the NAS during the confirmation window (transient timers do not survive reboot).

• Confirm SSH OK: Closes the current SSH connection and confirms only after a new successful connection for the matching, non-expired transaction — then the timer is cancelled.

• Rollback: Restores the last confirmed drop-in state or removes the file.

Important: For *high*, verify all clients support the chosen algorithms. Prefer low or middle for a first test.

15.12 UGOS core services

• Dropdown: Fixed core list of typical `*_serv` names (including `storage_serv`, `docker_serv`, `gateway_serv`, `miniscreen_serv`, `player_serv`, …). After each successful Update everything (sidebar), the app appends every other active unit ending in `_serv.service` (sorted, no duplicates).

• List from NAS: Reads active `*_serv` units over SSH and extends the combobox — useful without an immediate sidebar refresh.

• Show status: Short `systemctl` status for the selected unit in the log — quick check without a journal tail.

• Start / Stop / Restart: `systemctl` — may briefly interrupt services.

• Journal: Recent journal lines for the selected unit.

• UGOS service log (`.slog`): Refresh list fills the combo from `/var/ugreen/log/*.slog`; Show tail prints the last lines of the selected file (read-only). Many UGOS services write here in addition to `journalctl`.

• Support snapshot (read-only): Writes to the right-hand log e.g. `uname -a`, `/etc/os-release`, journal excerpt for `entry_serv`, tail excerpts from typical UGOS logs. No writes to NAS config.

15.13 Network (UGOS, read-only)

• Load summary: Read-only view of LAN1/LAN2 — UGOS JSON under `/etc/network/ugos.d/` (if readable) plus `ip link` and `ip addr` (link speed, connected/disconnected).

• No writes: IP profiles, bonding, or routing are not changed here. For runtime `ip` changes see Dashboard 42.3; permanent UGOS config stays in the NAS UI.

• Dual NIC: Both ports appear with label (e.g. LAN1/LAN2), interface name, and link state — helpful before NAS↔NAS transfers or when a port is unexpectedly down.

15.14 NGINX

• Reload: Runs UGOS reload (or `systemctl reload nginx`) and shows short status.

• Config recovery: After typing `RESTORE` in the dialog — copies ROM/default nginx into `/etc/nginx` and overwrites live config there. Custom edits can be lost — have backups.

15.15 earlyOOM

• Load / Save: Edits `/etc/default/earlyoom` and restarts the service. Parameter syntax matters — wrong settings can worsen OOM behavior.

15.16 Samba

1. Refresh shares — fills list from `testparm` / `smb.conf` (not `global` as target). 2. Empty recycle: Resolves share path and deletes contents of common recycle folders — irreversible for those files. 3. Quick share: Appends a simple block to `smb.conf`, validates with `testparm`, reloads `smbd`. Path must match a UGOS volume (e.g. `/volume1/…`).

15.17 LED & beeper

• Refresh LED slots, pick `diskN`, Identify: ~12 s blink to match bay to disk.

• Beeper: Short test tone via UGOS tool (model-dependent).

From the area: 49. Health complete: All blocks working together

Health is a diagnostic center, not just a display.

49.1 Main buttons

• Refresh = overall condition

• RAID/SMART/Storage = partial analysis

• Scheduler inventory = quick check of scheduled jobs and status

• Save report = documentation

49.2 System buttons

• Restart

- Shutdown

Only during the scheduled maintenance window.

49.3 UGOS Dashboard

Quick view of core services — plus the UGOS API live block (see 31.2): sysinfo, fan, network links, volumes. If the API is down, SSH checks (RAID, SMART, systemd failed) still run.

49.4 Telegram Instant Monitor

Useful for local testing without full watcher deployment.

49.5 NAS Watcher

Configuration + Deploy + Test in the same area.

49.6 Daily Report

Daily status report (not an alarm replacement, but a history).

From the area: 59. Practical instructions: Telegram alerting really robust

Many setups fail not because of the bot, but because of peripheral conditions. This complete sequence ensures a robust result:

1. Create bot (`@BotFather`, `/newbot`). 2. Secure tokens (do not post in screenshots). 3. Define target chat (individual chat or group). 4. Determine chat ID via `getUpdates`. 5. Enter token/chat ID in Settings. 6. Save. 7. Run Telegram test in the Health tab. 8. Confirm test message. 9. Deploy NAS Watcher. 10. Send watcher test. 11. Send Daily Report Test. 12. Check in the next 24 hours whether periodic messages arrive.

If individual steps fail, never do everything again immediately. Always keep the latest working version as a reference.

From the area: 66. Detailed catalog Health-Watcher switch

Each switch defines whether a specific check should be active in the Watcher.

- UPS check: NUT status
- UGOS core services: core services
- SMART daemon: smartd active/enabled
- Network ready: wait-online service
- File services: SMB/NFS/wsdd2
- Maintenance timers: trim/sysstat/logrotate/backups
- RAID checks: mdstat/mdcheck
- Docker runtime checks

Recommendation:

First activate the security and availability critical checks. Then expand gradually to avoid a flood of alarms.

---

16. Tab Storage & Sharing complete

From range: 32. Full reference: Storage tab

Buttons:

- Volumes/df
- Shares
- Refresh all
- Top20
- Disk scan
- Image to PC
- Image to NAS
- Restore from PC
- Restore from NAS

Fields:

- Top path
- Device combo
- Remote Image Path

From the range: 50. Storage complete

50.1 Volumes/Shares

Row of buttons for status recording.

50.2 Top 20 folders

Finds the largest subfolders under a path (typically `/volume1`). Can take up to ~5 minutes — from version 23.8.5 runs in the background; the app stays responsive.

Short: Storage tab → enter path → Top 20 folders → result appears in the log below.

Detailed step-by-step guide: §80.

50.3 Disk Image Operations

Strong tool for special cases.

Before each image/restore action:

1. Clearly select device. 2. Check target path. 3. plan enough memory/time.

50.4 Pools (UGOS API)

Button Pools (UGOS API) (also included in Refresh all): Reads pools, volumes, and physical disks via the UGOS web API — same view as the NAS UI.

Output includes:

- Pool name, RAID level, usage, sync/rebuild hints, pool-100% allocation note.
- Volume list per pool with usage percent.
- Physical disks with slot, model, temperature, short SMART status.

Prerequisite: UGOS API in Settings (port 9443, HTTPS, credentials). On failure: message in the log; SSH buttons Volumes/df and Shares still work independently.

Dashboard short view of the same API data: §6, 42.7.

17. Tab ACL complete

From range: 33. Full reference: ACL tab

Buttons:

- Ads (stat)
- UGACL info
- chmod 755
- chmod 777 rec
- Apply chmod
- apply chown
- Users
- Groups

Fields:

- Target path
- chmod mode
- chown value

From the range: 51. ACL complete

51.1 Diagnosis first

- Show
- UGACL info

51.2 Changes thereafter

- chmod 755 / 777 rec / custom
- chown apply

Rule:

- First test small, then roll out large.

---

18. Snapshots tab complete

From range: 34. Full reference: Snapshots tab

Buttons:

- detect backend
- btrfs list
- zfs list
- snapper list

- btrfs create
- zfs create
- snapper create
- btrfs delete
- zfs delete
- snapper delete

Field:

- Base path

From the range: 52. Snapshots complete

52.1 Detect backend

Necessary to select appropriate commands.

52.2 Lists and Creating

Before creating:

- Check base path.

52.3 Delete

Only with clear identification of the snapshot and fallback plan.

---

19. Tab Backup complete (Backup + Restore + Schedules together)

From Range: 35. Full Reference: Backup Tab

35.1 Backup Block Buttons

- Docker+Scripts Backup
- User data backup
- All data backup
- Refresh Lists
- Migration assistant (rsync for volume/NAS moves; also on NAS ↔ NAS tab)

Before a run, confirm that this creates a data archive: no database export, no filesystem snapshot, and no stopped applications. Running Docker containers with a writable mount on the source block success. Other writers are not detected. Success requires the archive to pass the app's own restore check; hardlinks are stored as regular file contents. Details: [backup/restore contract](BACKUP_RESTORE_CONTRACT_DE.md) and [operating limits](NAS_ACCEPTANCE_DE.md).

35.1a Migration assistant

Scenarios: same NAS (other volumes), push to another NAS, pull from another NAS, Synology/QNAP template. Generates a Bash script with `rsync -aHAX` (dry-run by default). Save to `/volume1/scripts/ugreen_migration_rsync.sh`. For interactive copy use NAS ↔ NAS.

Dialog buttons: Refresh script · Pre-flight check · Save on NAS · Run on NAS · NAS ↔ NAS tab · Close. Long runs (rsync, save) do not freeze the UI — buttons are briefly disabled while running.

Detailed step-by-step guide: §78. SSH timeouts: §79.

35.2 Backup Fields

- Scope combo
- Volume combo
- User combo
- Dest mode combo
- NAS profile combo
- PCpath
- USB combo
- optional remove-on-copy checkbox

35.3 Restore block

Fields:

- Source mode (`nas`/`pc`)
- Archive path
- Target path

Buttons:

- Select file
- Start restore

Security: TAR recovery requires a new, nonexistent target directory under an existing parent. Only the fully extracted result is published under that name. Existing data is never replaced. The recovery directory is initially private; check its contents and permissions before adopting files. Archive ACLs/xattrs, links and special files are unsupported. See [isolated recovery and acceptance limits (German)](ISOLATED_RESTORE_DE.md).

35.4 Scheduled Backup

Buttons:

- Load from NAS
- Save to NAS
- Remove
- Create/Update

Fields:

- Job label
- Job type
- Cron fields
- Additional options

Security (from 23.8.57 / extended 23.8.69): Saving to the NAS preflights sources/mounts and validates cron lines before any root write. State/runner live under `/var/lib/ugreen-nas-admin/`. Backup generations activate privately under a NAS-side lock (atomic cron enable). There is no automatic archive deletion — manage free space manually. Non-empty schedules ask for the same archive limit as a

manual backup before they are saved. After this change, write schedules to the NAS once. Detected Docker writers abort the run; applications are not stopped.

From the range: 53. Backup complete

53.1 Scope/Volume/User

Controls what is backed up.

53.2 Target mode

- NAS
- PC
- USB
- second NAS profile

53.3 Backup buttons

- Docker+Scripts
- User Data
- All Data

53.4 Restore

Fields:

- source mode
- source archive
- target path

Buttons:

- file picker
- restore start

53.5 Scheduled Backup

Complete job management with cron logic.

From the area: 61. Practical instructions: Backup and restore with verification

61.1 Create a full backup

1. Open backup tab. 2. Filter Scope/Volume/User. 3. Set target mode (NAS/PC/USB/second NAS). 4. Choose the appropriate backup button and confirm the data-archive notice. 5. Check the run. Success means the archive passes the app's own restore check and no running Docker writer covers the source. 6. Document archive path.

61.2 Perform a restore

1. Set source mode. 2. Select Source Archive. 3. Set the target path to a new folder that does not exist yet, under a parent that does. 4. Click 'Start Restore'. Existing data is not replaced. The new folder is initially private. 5. Result check:

- do files exist?
- Are rights correct?

- are services running again?

61.3 Follow-up inspection

- Health Refresh
- Test the affected app/container
- optional snapshot for a new stable stand

From the range: 78. Practical guide: Migration assistant (volume / NAS / Synology)

Goal: Move data in a controlled way — to another volume on the same UGREEN NAS, to a second NAS, or import from Synology/QNAP — without blind copying.

When to use which tool?

| Task | Recommendation | |------|----------------| | Individual folders/files via GUI between NAS and PC/peer | NAS ↔ NAS tab or Explorer | | Large, repeatable data migration with logging | Migration assistant (rsync script) | | Full backup as archive | Backup tab (tar/backup buttons) |

Prerequisites

1. SSH connection to the target UGREEN is working. 2. Enough free space at the destination (check Storage tab). 3. Snapshot or backup of source data (Snapshots or Backup tab) — before every live rsync. 4. For NAS↔NAS via SSH: SSH key on both systems (Settings → install SSH key) or password SSH. 5. Stop Docker containers if migrating Docker volumes (Docker tab → Stop).

Step 1 — Open the assistant

1. Backup tab → button Migration assistant, or 2. NAS ↔ NAS tab → same button in the left toolbar. 3. The Migration assistant window appears with scenario selection, path fields, and script preview below.

Step 2 — Choose scenario

Select one of four options:

1. Same NAS — different volume/path Example: Docker from `/volume1/docker` to `/volume2/docker`. rsync runs locally on the NAS (no remote host needed).

2. Copy to another NAS (push) Data from this UGREEN to another NAS. Enter remote host/IP and SSH user of the destination NAS.

3. Import from another NAS (pull) Data from another NAS to this UGREEN. Remote host/IP = source NAS, source path = path on source NAS.

4. From Synology/QNAP — template Like pull, but adjust paths manually for Synology/QNAP (e.g. `/volume1/photo` → `/volume1/photos` on UGREEN).

Step 3 — Enter paths

1. Source path: Absolute path with leading `/`, e.g. `/volume1/docker/jellyfin` or `/volume1/@home/user/Drive`. 2. Destination path: Absolute path on the NAS that receives data (for push: path on remote NAS). 3. Remote host/IP: Only for push, pull, or Synology/QNAP — IP or hostname of the other NAS. 4. SSH user: Usually `admin` or your NAS admin (same as SSH login).

Step 4 — Set options

1. Dry-run (-n) — on by default. Test first; no files are changed, only simulated. 2. --delete — enable only if the destination should exactly match the source (extra files at destination are removed). Caution — only after a successful dry-run.

Step 5 — Generate and review script

1. Click Refresh script (pre-filled on open). 2. Read the Bash script in the text area below:

- `rsync -aHAX --info=progress2 --numeric-ids`
- with dry-run: `-n` in the rsync line
- source and destination with trailing `/` (copies contents of the folder)
- variables `SRC=` / `DST=` / `RSYNC_SRC=` — Run on NAS executes the full script, not just the rsync line

3. Follow the checklist in the dialog: snapshot → stop apps → dry-run → live → update compose paths → start services.

Step 5a — All dialog buttons (reference)

| Button | Function | |--------|----------| | Refresh script | Rebuilds the Bash script from scenario, paths, and options | | Pre-flight check | Checks paths, space, and (if remote) SSH — result in text area + dialog; runs in background | | Save on NAS | Writes script to `/volume1/scripts/ugreen_migration_rsync.sh` (executable); background | | Run on NAS | Runs pre-flight, then the script via SSH on the NAS — background, may take very long | | NAS ↔ NAS tab | Closes assistant and switches to GUI copy tab | | Close | Close dialog without further run |

While a background job runs, action buttons are briefly disabled; the status line shows e.g. “Pre-flight check running…” or “Migration/rsync running on NAS…”.

Step 6 — Pre-flight check (recommended, also automatic before run)

1. Click Pre-flight check (or Run on NAS directly — pre-flight runs first automatically). 2. The app checks via SSH on UGREEN (and optionally SSH to remote NAS):

- Source path exists (`test -d`)
- Destination reachable (folder creatable)
- Source size (`du -sk`) and free space at destination (`df`)
- for push/pull/Synology: SSH to remote NAS (key from UGREEN to other NAS required — Settings §23.4)

3. Result in the text area under `=== Pre-flight check ===` — each line with `•`. 4. Dialog passed → continue with dry-run or live. Dialog failed → fix paths/key/space, check again.

What pre-flight reports

| Message | Meaning | Action | |---------|---------|--------| | Source path exists | OK | — | | Source path missing | Wrong or unmounted path | Check path in Explorer | | Destination reachable | OK | — | | Destination missing | No write access or wrong path | sudo/ACL, fix path | | Source size approx. … | Rough data amount | — | | Free space at destination approx. … | Space on target volume | Storage tab, other volume | | Warning: less free space than source | Migration likely to fail | Free space or other target | | SSH to remote NAS OK | Key/network OK | — | | SSH to remote NAS failed | No key, firewall, wrong host | Key on UGREEN for remote NAS, port 22 | | Remote host/IP missing | Push/pull with empty host | Enter IP |

Step 7 — Run dry-run on NAS

1. Ensure Dry-run is checked. 2. Click Refresh script (so `-n` is in the rsync line). 3. Click Run on NAS — pre-flight first, then rsync simulation. 4. Review output under `--- NAS ---` in the text area — what would be copied without writing. 5. On errors: read exit code and message; check §79 (timeout) and SSH key.

Step 8 — Live migration

1. Keep Docker/services that access the source stopped. 2. Uncheck Dry-run. 3. Click Refresh script — `-n` disappears from the rsync line. 4. Click Run on NAS — read and confirm the live-run dialog. 5. rsync may

take hours — keep window open; UI stays responsive (other tabs OK). Keep Long timeout at 0 in Settings (§79). 6. On success: “rsync complete” dialog; on failure: exit-code message — check log below.

Step 9 — Save script for later (optional)

1. Save on NAS — creates `/volume1/scripts/ugreen_migration_rsync.sh` (executable); runs in background. 2. For repeat or cron: Scripts tab → open file, adjust, schedule in planner.

Step 10 — After migration

1. Update compose/app paths if the volume changed (Docker wizard, paths in YAML). 2. Restart containers/services (Docker tab → Start). 3. Spot check: file counts at destination, open app, refresh Health tab. 4. Only when everything works: optionally delete source data (not immediately — keep a safety copy).

Example A: Docker from volume1 to volume2 (same NAS)

| Field | Value | |-------|-------| | Scenario | Same NAS — different volume | | Source path | `/volume1/docker` | | Destination path | `/volume2/docker` | | Dry-run | on → test → off for live |

Example B: Pull from Synology to UGREEN

| Field | Value | |-------|-------| | Scenario | Import from another NAS | | Source path | `/volume1/photo` (Synology shared folder) | | Destination path | `/volume1/photos` | | Remote host | `192.168.1.50` | | SSH user | `admin` |

Synology: enable SSH in Control Panel; path may be `/volume1/<share>` instead of `@home`.

GUI alternative

For smaller amounts without a script: assistant → NAS ↔ NAS tab — UGREEN left, peer right, select folders, copy (SMB). The migration assistant is meant for large, full tree copies with rsync.

Common issues

- Forgot to disable dry-run: Live run with `-n` copies nothing — uncheck, refresh script.
- Trailing slash: `/volume1/data/` copies contents; assistant sets `/` consistently.
- SSH to remote NAS failed: Key from UGREEN to remote NAS (Settings §23.4), firewall port 22.
- SSH command aborted (timeout): Default timeout too low — set Long (s) = 0 for rsync (§79).
- Command failed (exit …): Message includes exit code and excerpt — check paths, sudo, space.
- Pre-flight: less space at destination: Use another disk/volume or free space first.

From the range: 79. Practical guide: SSH command timeouts (Settings)

Goal: Short SSH actions should not hang forever; long jobs (rsync, `du`, backup) may run unlimited.

Where to configure

1. Tab Settings (□□). 2. Section Connection — row SSH command: below UGOS API. 3. Fields:

- Default (s) — timeout for normal commands (lists, health, Explorer `ls`, …).
- Long (s) — timeout for long runners; 0 = unlimited (default, recommended).

4. Click Save at the top (not only “Apply to current UI”) so values persist in `app_settings.json`.

Recommended values

| Situation | Default (s) | Long (s) | |-----------|-------------|----------| | Normal / homelab | 120 | 0 | | Very slow NAS / VPN | 180–300 | 0 | | Small commands only, never rsync in app | 60 | 3600 (1 h) |

Which actions use which timeout?

| Timeout | Examples in the app | |---------|---------------------| | Default | Explorer directory listing, Docker list, health single commands, pre-flight (~90 s fixed), save script | | Long | Migration assistant rsync, Storage Top 20, backup runs |

What happens on timeout?

- Message: "SSH command aborted (timeout)…"
- The app does not freeze (background threads for migration/Top-20).
- Connection may reconnect on the next command.
- Fix: Increase default or set Long = 0 for that long job.

Exit-code messages (from 23.8.5)

Failed commands (migration, Top-20) may show:

`Command failed (exit 23): …`

| Exit | Typical cause | |------|----------------| | 1 | General error (rsync, script) | | 2 | Syntax / script error | | 23 | rsync: partial transfer (space, permissions, interrupted) | | 255 | SSH connection dropped |

Always read the log text below the dialog.

From the range: 80. Practical guide: Storage Top-20 folders

Goal: Find which folders under `/volume1` (or another path) use the most space — e.g. before migration or cleanup.

Step 1 — Open Storage tab

1. Sidebar → Storage. 2. Scroll down if needed — Top 20 / path field area.

Step 2 — Set path

1. Field Path for Top-20 (e.g. `/volume1` or `/volume1/docker`). 2. Use paths that exist on the NAS; check Volumes (df -h) above.

Step 3 — Start analysis

1. Click Top 20 folders. 2. Log shows heading `=== TOP 20 (du under …) ===`. 3. Status line: "Calculating largest folders (background)…" — app stays responsive (from 23.8.5). 4. While running, another Top-20 click is ignored (no double start).

Step 4 — Read result

1. When done: status "Top 20 complete". 2. List sorted by size (largest first), typically depth 3. 3. Use sizes to pick candidates for archive, migration (§78), or cleanup.

Step 5 — Empty or error output

1. Read exit code or timeout message (§79). 2. Permission denied: app retries with sudo automatically — NAS user needs sudo. 3. Path too deep/large: use narrower path (e.g. `/volume1/docker` instead of `/volume1`). 4. Timeout: set Long (s) = 0 in Settings; Top-20 uses long timeout.

Note: `du` on very large trees can take several minutes (NAS-side `timeout 300`). Be patient or use a smaller start path.

From the area: 67. Detailed catalog backup-restore fields

67.1 Source Mode

Defines from which context the archive source comes:

- NAS file system
- local file system
- USB medium

67.2 Source Archives

Path to the specific backup file (e.g. tar.gz). Must be accessible for the selected source mode.

67.3 Target Path

Restore target on NAS.

Important:

- Rights available?
- enough space?
- Target empty/overwritable?

From the area: 76. Full workflow D: Backup strategy for multiple targets

The goal is to not just have a backup, but a usable recovery concept.

Strategy:

1. Primary target NAS internal (fast). 2. Secondary objective second NAS (risk separation). 3. Optional USB/offline for emergencies.

Implementation in the app:

1. Complete settings with second NAS. 2. Backup tab Select target mode for each run. 3. Set daily/weekly jobs in the scheduler. 4. Monthly restore test on test path.

Rule for real security:

A backup is only considered reliable once a restore has been tested successfully.

---

20. Info dialog complete

From the area: 54. Info dialog complete

Document buttons:

- README
- manual
- CHANGELOG

Other elements:

- YouTube
- Support link
- Email contact
- About text

---

21. Overall operations, checklists, troubleshooting

From the range: 19. Complete operation order (recommended)

1. Settings complete. 2. Check connection. 3. Dashboard + Health. 4. Building Docker services. 5. Test scripts, then scheduler. 6. Set backup targets. 7. Restore test in test path. 8. Deploy Watcher and Daily.

From area: 20. Bug fixes by area

20.1 Connection

- SSH badge red: Check IP/Port/User/Auth.
- Key active, but password in the field: standardize auth path.

20.2 Docker

- List empty despite expected container: `List` first.
- Start fails: Check Inspect + Logs + Runtime in Health.

20.3 Backup/Restore

- Target profile missing: Check settings second profile.
- Restore without effect: Check archive path/mode/destination path.

20.4 Watcher/Daily

- Deploy ok, but no messages: check channel credentials and triggers.
- Too many messages: Adjust thresholds/checkboxes.

20.5 ACL/Explorer

- Unexpected rights effects: first read Stat/UGACL, then specifically change it.
- Deletion error: Check focus tree and path.

From the area: 55. Checklists (operational)

55.1 Before critical change

1. Health Refresh 2. Snapshot/Backup 3. Individual change 4. Check logs/status 5. if necessary next change

55.2 Before Docker update

1. Current list 2. Check selection 3. Logs baseline 4. Update 5. Functional test

55.3 Before Restore

1. Check archive 2. Check target path 3. Save previous state 4. Restore 5. Check integrity

---

From the range: 56. Large error patterns and clean response

56.1 “Everything seems broken”

Don't click everywhere. Sequence:

1. SSH 2. Health Refresh 3. Core Services 4. Storage 5. Docker Runtime

56.2 “Just one service broken”

1. Open the affected tab 2. Update list/status 3. Logs/Inspect 4. targeted correction

56.3 “Notifications are not coming”

1. Settings Credentials 2. Channel selection 3. Test buttons 4. NAS network

---

From the area: 58. Practical instructions: First commissioning without gaps

These instructions will take you from the first start to the stable basic configuration without skipping any areas.

1. Start the app and select the theme/language so that you can immediately see the interface in your work context. 2. Open Settings and enter the connection data to the UGREEN NAS. 3. Test the SSH connection so that it is clear that the base is up and running. 4. Set a screenshot directory so that you can create documentation immediately if necessary. 5. Enter Telegram and/or SMTP and check with test. 6. Enter a second NAS profile if you want to use NAS-to-NAS transfers or second destinations for backups. 7. Save and then refresh the relevant tabs immediately afterwards. 8. Open the Health tab and run “Refresh” completely. 9. Check dashboard: load, volumes, docker, fan status. 10. Open the Scripts tab and make a backup of an existing script base.

If these ten points have been completed properly, the app is usually ready for everyday use and maintenance.

From the area: 68. Safety rules for productive operation

1. Always take a health snapshot before major changes. 2. Always backup/snapshot before delete/restore actions. 3. Never touch multiple critical areas at the same time (e.g. Docker + ACL + Storage in one step). 4. Make changes sequentially and verifiably. 5. If you are unsure, test in a smaller scope first.

These rules reduce downtime and make sources of errors traceable.

From the range: 69. Complete troubleshooting matrix

69.1 SSH does not connect

Symptom:

- No data in Dashboard/Health.

Test:

- Host/port correct?
- Network accessible?
- User/pass correct?
- SSH active on NAS?

69.2 Docker shows nothing

Symptom:

- Empty list or error text.

Test:

- Docker service running?
- User has necessary rights?
- Host communication stable?

69.3 Backup to second NAS missing

Symptom:

- No profile in selection.

Test:

- Settings saved for second NAS?
- Fields complete?
- Backup sources updated?

69.4 Telegram does not report

Symptom:

- No message during test.

Test:

- Token/Chat ID
- Network outbound
- Channel selection in Watcher/Daily

69.5 Fan profile reacts unexpectedly

Symptom:

- RPM does not jump as expected.

Test:

- selected mode correct?
- Pressed apply?
- UGOS return executed if conflict?

From the range: 70. Maintenance plan for stable long-term use

Daily:

- Dashboard quick check
- Read alerts

Weekly:

- Health full refresh
- Check Docker/Storage abnormalities

Monthly:

- Backup-restore test in a small scope
- Review cron jobs and script versions
- Test notification channels

Quarterly:

- Clean up rights/ACL concept
- Update snapshot/backup strategy

- Compare documentation with current status

From the area: 71. Glossary in operational language

- UGOS: UGREEN NAS operating system.
- Watcher: Running test process with alarm output.
- Daily Report: Daily summary.
- Cron: Scheduled job execution.
- SMART: Drive diagnostics.
- mdcheck: RAID checking mechanism.
- NUT: UPS management service.
- SMB/NFS: File sharing protocols.
- Inspect: Detailed container configuration.
- Snapshot: Snapshot of the file system state.

From the area: 73. Full workflow A: Completely set up the new NAS

This workflow describes a complete initial commissioning including monitoring and backup.

Phase 1 - Connection:

1. Fill out the settings (SSH, User, Auth, Sudo). 2. Test connection. 3. Save.

Phase 2 - Visibility:

1. Load dashboard. 2. Health Refresh. 3. Check model display.

Phase 3 - Notification:

1. Set up Telegram (token/chat ID). 2. Set up SMTP (optional additionally). 3. Send test from Health.

Phase 4 - Operational Functions:

1. Save scripts. 2. Create scheduler test job. 3. Check Docker list. 4. Capture storage/ACL snapshot.

Phase 5 - Data Backup:

1. Set backup destination. 2. Start initial backup. 3. Check restore in small scope.

Phase 6 - continuous operation:

1. Deploy Watcher. 2. Enable Daily Report. 3. Apply maintenance plan according to Chapter 70.

From the area: 74. Full workflow B: Resolve disruptions in production in a structured manner

Example: Services react with a delay, load increases, users report failures.

Step 1 - Situation picture:

- Read dashboard (CPU, RAM, network, volumes).
- Health Refresh.

Step 2 - Limitation:

- Check Docker Runtime status.
- Check core services.

- Check RAID/SMART.

Step 3 - Verification:

- For Docker problems: Logs + Inspect.
- For storage problems: Top20 + Volumes + Shares.
- For script problems: last cron jobs, test run manually.

Step 4 - Correction:

- Just one change at a time.
- Check the direct effect after each change.

Step 5 - Hedging:

- Snapshot/backup after stabilization.
- Document the cause briefly (internally or in the changelog).

This process prevents activism and reduces secondary errors.

From the range: 75. Full workflow C: Planned maintenance without surprise failures

Preparation:

1. Define maintenance windows. 2. Capture affected services/containers. 3. Create backup/snapshot.

Implementation:

1. Selectively update Docker. 2. Check scripts/cron jobs. 3. Rights adjustments only targeted. 4. Update health after every big step.

Acceptance:

1. Core function test (user view). 2. Telegram/email test message. 3. Check performance baseline.

Rework:

1. note open points. 2. naechstes Wartungsfenster vorbereiten.

---

22. Graduation

From the area: 21. End

This manual is intended as a complete operating basis for the current app. If you use the sequences described and work with the respective safety rules for each area, you can use the app productively in a stable and controlled manner.

From the range: 38. End note

This reference is intentionally complete and formulated specifically for each area. If you use it as a workflow, the app can be operated in a stable and reproducible manner.

From the area: 57. Graduation

This manual maps the current UI and functional logic and describes the app as a complete operational workflow. Use it as a reference when working, not just in case of errors. This means that operation and changes remain comprehensible and stable.

From the area: 72. Conclusion

The app is designed as an operations center: configure, check, change, verify. This manual is therefore not short, but written as a complete working reference. If you consistently apply the processes described, you will get reproducible results, fewer failures and significantly faster troubleshooting.
